

Predictor Test Suggestions

总体判断：这份测试计划可以按照它执行，而且方向是正确的。

它已经把 predictor 的测试边界限定在 advisory query 本身，不把 AL/IM/PI cost、URG、PointCloud2、A*、B-spline、planner success 混进来，这是正确边界。计划中也已经明确 probe 只订阅 odom、map cloud、GNSS、`/iap/integrity`，构造 `IntegritySnapshot` 后调用 `PredictorModule::query()`，并用 CSV/JSON 验证，不依赖 planner 或 monitor fusion 输出。

不过我建议你在执行前补 5 类内容，否则实验结果会“能通过 validator”，但不够支撑论文/汇报里的 predictor 正确性结论。

1. 需要补充的关键建议

1.1 每次实验必须保存 run manifest

现在计划说会输出：

代码块

```
1 test_predictor_query_probe.csv
2 test_predictor_query_probe_summary.json
3 test_predictor_validation_summary.json
```

这个还不够。建议每个实验目录额外保存：

代码块

```
1 run_manifest.json
```

里面记录：

代码块

```
1 git_commit
2 branch
3 build_type
4 experiment_name
5 araim_sim_preset
```

```
6 predictor_output_mode
7 fusion_mode
8 start_time
9 duration_s
10 random_seed
11 map_seed
12 gnss_seed
13 lidar_seed
14 predictor_params
15 validator_params
16 launch_arguments
17 machine_name
18 cpu_model
```

否则你之后很难复现实验，尤其是 latency、fallback ratio、HPL/VPL median。

1.2 validator 不应只判断 pass/fail，还要判断趋势

当前 acceptance criteria 已经包括 query count、finite PL、fallback reason、selected source、`lambda_sum_error`、latency p50/p95/max 等。

但 predictor 最重要的是 **趋势正确**，建议补充 trend checks：

代码块

```
1 GNSS degraded scenario:
2   degraded-region median HPL > open-sky median HPL
3
4 GNSS outage scenario:
5   outage-window valid ratio drops
6   fallback reason histogram includes no_gnss_epoch or too_few_sats
7
8 LiDAR sparse scenario:
9   lidar_n_primitives decreases
10  lidar_condition or degeneracy_score worsens
11
12 Fusion nominal:
13   lambda_fused_trace  $\approx$  lambda_prior_trace + lambda_gnss_trace +
14   lambda_lidar_trace
15   fused result finite and source contribution explainable
```

也就是说，validator 不只是检查数值存在，还要检查“场景变化是否导致合理响应”。

1.3 必须补 current vs advisory separation 图

你的 predictor 不能被误解成 current ARAIM 的复制品。建议每个实验都保存：

代码块

```
1 current_hpl vs selected_hpl scatter
2 current_vpl vs selected_vpl scatter
3 current_hpl / selected_hpl timeline
4 current_vpl / selected_vpl timeline
```

目标不是要求二者完全不同，而是证明：

代码块

```
1 Predictor uses current ARAIM only as snapshot/context.
2 It does not blindly copy current HPL/VPL as future advisory PL.
```

尤其是 `predictor_gnss_open_sky_only` 和 `predictor_gnss_degraded_lidar_good` 必须画这个图。

1.4 Fusion 实验必须保存 source contribution 图

你计划里有 `gnss_only`、`lidar_only`、`fusion` 三种 output mode，并且会记录 `lambda_sum_error`。这个很好。

但 fusion 是否可信，不能只看 selected source 是 `FUSION`。还要证明 fusion 里的每个 source 贡献合理。

建议每个 fusion 实验保存：

代码块

```
1 lambda_prior_trace timeline
2 lambda_gnss_trace timeline
3 lambda_lidar_trace timeline
4 lambda_fused_trace timeline
5 lambda_sum_error timeline
6 lambda_fused_min_eig timeline
7 lambda_fused_condition timeline
```

并保存一个 stacked plot：

代码块

- 1 information contribution stack:
- 2 prior / GNSS / LiDAR trace ratio over time

这样你才能解释：

代码块

- 1 为什么这个位置 fused PL 变小？
- 2 是 GNSS 强了，LiDAR 强了，还是 prior 过强？

1.5 LiDAR 暂时只做 diagnostic，不要急着用它证明安全

你的计划包含：

代码块

- 1 predictor_lidar_feature_rich_only
- 2 predictor_lidar_sparse_gnss_good
- 3 predictor_gnss_outage_lidar_recovery

这些实验可以跑，但我建议你在报告中把 LiDAR 结论分成两级：

代码块

- 1 Level 1: LiDAR diagnostic validity
- 2 primitive count, condition, TDOP, alpha, fallback reason 是否合理
- 3
- 4 Level 2: LiDAR fusion validity
- 5 LiDAR information 是否真的应该降低 fused PL

现在优先接受 Level 1。Level 2 需要 corridor degeneracy 通过后再作为强 claim。

2. 每个实验应该补充哪些图片

下面按你计划里的 7 个 system experiments 给出建议。

E1. predictor_gnss_open_sky_only

目的

验证 GNSS-only predictor 在正常 open-sky 下稳定、finite、低 fallback。

必须保存图片

代码块

```
1  fig_E1_hpl_vpl_timeline.png
```

内容：

代码块

```
1  time vs gnss_hpl
2  time vs gnss_vpl
3  time vs selected_hpl
4  time vs selected_vpl
```

代码块

```
1  fig_E1_gnss_geometry_timeline.png
```

内容：

代码块

```
1  time vs gnss_n_visible
2  time vs gnss_pdop
3  time vs gnss_condition
```

代码块

```
1  fig_E1_current_vs_advisory_scatter.png
```

内容：

代码块

```
1  x-axis: current_hpl
2  y-axis: selected_hpl
3  另一个子图: current_vpl vs selected_vpl
```

代码块

```
1 fig_E1_latency_histogram.png
```

内容：

代码块

```
1 gnss_us
2 fusion_us
3 module_total_us
```

必须保存 debug 数据

代码块

```
1 predictor_query_probe.csv
2 summary.json
3 validation_summary.json
4 run_manifest.json
5 gnss_satellite_visibility.csv
6 predictor_params.yaml/json
```

通过解释

你要能说：

代码块

```
1 open-sky 下 GNSS predictor 输出 finite, n_visible 稳定, PDOP 低, fallback ratio 低, latency 稳定。
```

E2. predictor_lidar_feature_rich_only

目的

验证 LiDAR FIM diagnostic 在 feature-rich 环境下有效。

必须保存图片

代码块

```
1  fig_E2_lidar_fim_timeline.png
```

内容:

代码块

```
1  time vs lidar_n_primitives
2  time vs lidar_alpha
3  time vs lidar_tdop
4  time vs lidar_condition
```

代码块

```
1  fig_E2_lidar_eigen_timeline.png
```

内容:

代码块

```
1  time vs lidar_min_eig
2  time vs lidar_max_eig
3  time vs lidar_condition
```

代码块

```
1  fig_E2_query_points_and_primitives_topdown.png
```

内容:

代码块

```
1  map cloud / feature primitives
2  UAV trajectory
3  query points
```

代码块

```
1  fig_E2_lidar_validity_histogram.png
```

内容:

代码块

```
1 lidar_valid count
2 fallback reason histogram
3 selected source count
```

必须保存 debug 数据

代码块

```
1 lidar_primitives_debug.csv
2 lidar_fim_matrix_debug.csv
3 lidar_fallback_reason_histogram.json
4 local_map_snapshot.pcd 或 downsampled_map.csv
```

通过解释

你要能说：

代码块

```
1 feature-rich 区域 primitive 数量足够, LiDAR FIM 有效, condition 不病态, fallback 少。
```

E3. predictor_fusion_nominal

目的

验证 nominal fusion 路径数学闭合。

必须保存图片

代码块

```
1 fig_E3_source_hpl_vpl_timeline.png
```

内容：

代码块

```
1 gnss_hpl / lidar_hpl / fused_hpl / selected_hpl
```



```
2 gnss_vpl / lidar_vpl / fused_vpl / selected_vpl
```

代码块

```
1 fig_E3_lambda_contribution_timeline.png
```

内容：

代码块

```
1 lambda_prior_trace
2 lambda_gnss_trace
3 lambda_lidar_trace
4 lambda_fused_trace
```

代码块

```
1 fig_E3_lambda_sum_error_timeline.png
```

内容：

代码块

```
1 lambda_sum_error
2 validator threshold
```

代码块

```
1 fig_E3_fusion_condition_timeline.png
```

内容：

代码块

```
1 lambda_fused_min_eig
2 lambda_fused_condition
```

必须保存 debug 数据

代码块

```
1  lambda_prior_3x3.csv
2  lambda_gnss_3x3.csv
3  lambda_lidar_3x3.csv
4  lambda_fused_3x3.csv
5  fusion_diagnostics.csv
6  source_selection_debug.csv
```

通过解释

你要能说：

代码块

```
1  fusion 满足  $\lambda_{pred} = \lambda_{prior} + \lambda_{gnss} + \lambda_{lidar}$ ;
2  fused matrix SPD;
3  lambda_sum_error 在阈值内;
4  source contribution 可解释。
```

E4. predictor_gnss_degraded_lidar_good

目的

验证 GNSS degraded 时 predictor 能反映 GNSS 风险，同时 LiDAR diagnostic/fusion 有解释力。

必须保存图片

代码块

```
1  fig_E4_gnss_degradation_timeline.png
```

内容：

代码块

```
1  gnss_n_visible
2  gnss_pdop
3  gnss_hpl
4  gnss_vpl
5  fallback_reason markers
```

代码块

```
1  fig_E4_lidar_support_timeline.png
```

内容:

代码块

```
1  lidar_n_primitives
2  lidar_condition
3  lidar_alpha
4  lambda_lidar_trace
```

代码块

```
1  fig_E4_gnss_vs_fusion_hpl.png
```

内容:

代码块

```
1  gnss_hpl vs fused_hpl over time
2  gnss_vpl vs fused_vpl over time
```

代码块

```
1  fig_E4_spatial_query_map.png
```

内容:

代码块

```
1  UAV trajectory
2  query points colored by selected_hpl
3  GNSS degraded region boundary if available
```

必须保存 debug 数据

代码块

```
1  gnss_visibility_by_query.csv
2  satellite_used_mask.csv
3  lidar_fusion_gate.csv
```

```
4 source_contribution.csv
5 fallback_reason_histogram.json
```

通过解释

你要能说：

代码块

```
1 GNSS degraded 段 n_visible 下降 / PDOP 上升 / GNSS PL 上升；
2 LiDAR good 段 diagnostic 保持有效；
3 fusion 是否降低 PL 有明确 source contribution 支撑。
```

如果你目前默认 GNSS-only，则这组实验仍可跑，但报告应写成：

代码块

```
1 LiDAR good is logged as diagnostic; it does not yet modify selected PL in GNSS-
  only mode.
```

E5. predictor_lidar_sparse_gnss_good

目的

验证 LiDAR 退化不会污染 GNSS-good 输出。

必须保存图片

代码块

```
1 fig_E5_lidar_degradation_timeline.png
```

内容：

代码块

```
1 lidar_n_primitives
2 lidar_condition
3 lidar_tdop
4 lidar_fallback flag
```

代码块 fig_E5_gnss_stability_timeline.png

内容：

代码块

```
1 gnss_n_visible
2 gnss_pdop
3 gnss_hpl
4 gnss_vpl
```

代码块

```
1 fig_E5_fusion_gate_timeline.png
```

内容：

代码块

```
1 lidar_allowed_for_fusion
2 lidar_fusion_gate_reason
3 selected_source
```

代码块

```
1 fig_E5_selected_vs_gnss_pl.png
```

内容：

代码块

```
1 selected_hpl vs gnss_hpl
2 selected_vpl vs gnss_vpl
```

必须保存 debug 数据

代码块

```
1 lidar_fallback_reason_histogram.json
2 lidar_gate_debug.csv
```

通过解释

你要能说：

代码块

- 1 LiDAR sparse/degenerate 时 predictor 不应因为错误 LiDAR information 让 selected PL 变得更乐观。

这是非常关键的 safety 结论。

E6. predictor_gnss_outage_lidar_recovery

目的

验证 GNSS outage 时 fallback reason 明确，fusion/recovery 行为清楚。

必须保存图片

代码块

- 1 fig_E6_outage_window_timeline.png

内容：

代码块

- 1 gnss_valid
- 2 gnss_n_visible
- 3 selected_valid
- 4 selected_fallback
- 5 selected_hpl/vpl

并用背景阴影标出 outage window。

代码块

- 1 fig_E6_fallback_reason_timeline.png

内容：

代码块

```
1 time vs fallback_reason category
```

代码块

```
1 fig_E6_recovery_latency.png
```

内容：

代码块

```
1 GNSS restored time
2 predictor valid restored time
3 recovery delay
```

代码块

```
1 fig_E6_source_selection_timeline.png
```

内容：

代码块

```
1 selected_source over time
2 GNSS / LIDAR / FUSION / INVALID
```

必须保存 debug 数据

代码块

```
1 outage_event_times.json
2 fallback_reason_by_time.csv
3 source_selection_debug.csv
4 recovery_metrics.json
```

通过解释

你要能说：

代码块

```
1 gnss outage 不会被静默当成安全；
2 fallback reason 非空；
3 恢复后 predictor valid 恢复延迟可量化。
```

如果 Stage 1 不允许 LiDAR-only 接管，那么 outage 时 selected 应该 invalid，而不是强行返回 LiDAR recovery。

E7. predictor_no_source_negative

目的

验证没有 advisory source 时不会产生 fake safe output。

必须保存图片

代码块

```
1 fig_E7_valid_fallback_flags.png
```

内容：

代码块

```
1 selected_valid
2 selected_fallback
3 gnss_valid
4 lidar_valid
5 fused_valid
```

代码块

```
1 fig_E7_fallback_reason_histogram.png
```

内容：

代码块

```
1 fallback reason counts
```

代码块


```
1  fig_E7_selected_pl_timeline.png
```

内容：

代码块

```
1  selected_hpl/vpl
2  标注 invalid rows
```

必须保存 debug 数据

代码块

```
1  negative_case_rows.csv
2  fallback_reason_histogram.json
3  invalid_output_audit.json
```

通过解释

你要能说：

代码块

```
1  no source case 下 predictor 明确 invalid/fallback，不输出看似可用的 safe PL。
```

3. 所有实验都应该统一保存的图片

每个实验目录至少保存这 8 张图：

代码块

```
1  01_selected_pl_timeline.png
2  02_source_validity_timeline.png
3  03_source_selection_histogram.png
4  04_fallback_reason_histogram.png
5  05_gnss_geometry_timeline.png
6  06_lidar_diagnostics_timeline.png
7  07_lambda_contribution_timeline.png
8  08_latency_distribution.png
```

如果某个实验不启用 LiDAR 或 fusion，对应图可以为空但要标注：

代码块

```
1 not enabled in this run
```

这样后面汇报时版式统一。

4. 所有实验都应该保存的 debug 数据

建议每个实验目录结构固定成：

代码块

```
1 predictor_experiment_<name>/
2   run_manifest.json
3   test_predictor_query_probe.csv
4   test_predictor_query_probe_summary.json
5   test_predictor_validation_summary.json
6
7   params/
8     predictor_params.yaml
9     launch_args.yaml
10    araim_sim_preset.yaml
11
12   debug/
13     snapshot_debug.csv
14     gnss_epoch_debug.csv
15     gnss_visibility_by_query.csv
16     lidar_primitives_debug.csv
17     lidar_fim_debug.csv
18     fusion_matrix_debug.csv
19     source_selection_debug.csv
20     fallback_reason_by_time.csv
21
22   figures/
23     01_selected_pl_timeline.png
24     02_source_validity_timeline.png
25     03_source_selection_histogram.png
26     04_fallback_reason_histogram.png
27     05_gnss_geometry_timeline.png
28     06_lidar_diagnostics_timeline.png
29     07_lambda_contribution_timeline.png
30     08_latency_distribution.png
```

```
31
32     optional_bag/
33     short_replay.bag
```

其中最重要的是：

代码块

```
1  source_selection_debug.csv
2  fusion_matrix_debug.csv
3  fallback_reason_by_time.csv
```

因为这些能解释 predictor 为什么输出某个 PL。

5. 建议补充的 CSV 字段

你现有计划已经记录 timing、source flags、fallback reasons、matrix diagnostics、valid/fallback、`lambda_sum_error`。

我建议再补这些字段：

5.1 Snapshot freshness

代码块

```
1  odom_stamp_s
2  integrity_stamp_s
3  gnss_epoch_stamp_s
4  snapshot_stamp_s
5  odom_age_s
6  integrity_age_s
7  gnss_age_s
8  snapshot_age_s
```

5.2 Current/advisory separation

代码块

```
1  current_hpl
2  current_vpl
3  current_pl_e
4  current_pl_n
5  current_pl_u
```

```
6 selected_hpl
7 selected_vpl
8 selected_pl_e
9 selected_pl_n
10 selected_pl_u
11 copied_current_flag
```

`copied_current_flag` 可以简单定义为：

代码块

```
1 abs(selected_hpl - current_hpl) < eps && output_mode != constant_current
```

用于抓 bug。

5.3 Query spatial info

代码块

```
1 query_x
2 query_y
3 query_z
4 query_offset_x
5 query_offset_y
6 query_offset_z
7 query_label
8 scenario_region_label
```

`scenario_region_label` 可以是：

代码块

```
1 open_sky
2 canopy
3 corridor
4 outage
5 unknown
```

有了这个字段，你才能画分区域 boxplot。

5.4 GNSS details

代码块

```
1  sat_used_mask
2  sat_visible_mask
3  excluded_prn_mask
4  n_excluded
5  min_elevation_deg
6  mean_elevation_deg
7  effective_sigma_mean
8  effective_sigma_max
```

5.5 LiDAR degeneracy details

代码块

```
1  normal_diversity_score
2  corridor_axis_info
3  along_corridor_sigma
4  lidar_degeneracy_score
5  lidar_gate_passed
6  lidar_gate_reason
```

考虑到你之前 corridor LiDAR PL 不能反映退化，这些字段非常必要。

6. 建议补充的通过标准

在原有 acceptance criteria 上补下面这些：

6.1 安全 fallback 标准

代码块

```
1  No-source / no-GNSS / too-few-sats rows:
2      selected_valid must be false
3      or selected_fallback must be true
4      fallback_reason must be non-empty
5      selected_source must not be silently FUSION
```

6.2 趋势标准

代码块

```
1 GNSS degraded scenario:
2   median(gnss_hpl_degraded) > median(gnss_hpl_open)
3   median(gnss_pdop_degraded) > median(gnss_pdop_open)
4
5 LiDAR sparse scenario:
6   median(lidar_condition_sparse) > median(lidar_condition_feature_rich)
7   or median(lidar_n_primitives_sparse) <
   median(lidar_n_primitives_feature_rich)
```

6.3 Fusion 数学标准

代码块

```
1 lambda_sum_error <= threshold
2 lambda_fused_min_eig > eps
3 lambda_fused_condition < condition_max
4 fused PL finite for valid rows
```

6.4 不复制 current PL 标准

代码块

```
1 In advisory modes:
2   copied_current_flag ratio < 5%
```

如果超过这个比例，需要人工检查是不是场景本来相近，还是代码真的复制 current PL。

7. 执行顺序建议

可以按照你的计划执行，但建议顺序改成：

代码块

```
1 Step 1. 先跑 isolated GTest
2 Step 2. 跑 predictor_gnss_open_sky_only
3 Step 3. 跑 predictor_no_source_negative
4 Step 4. 跑 predictor_gnss_degraded_lidar_good
5 Step 5. 跑 predictor_lidar_sparse_gnss_good
```

- 6 Step 6. 跑 predictor_fusion_nominal
- 7 Step 7. 跑 predictor_lidar_feature_rich_only
- 8 Step 8. 跑 predictor_gnss_outage_lidar_recovery
- 9 Step 9. 每个关键实验重复两次, 做 reproducibility comparison

我会把 negative test 提前, 因为它能最快发现最危险的问题: **unknown 被伪装成 safe finite PL**。

8. 最终建议

这份计划 **可以执行**, 但执行前建议补上:

代码块

- 1 1. run_manifest.json
- 2 2. current vs advisory separation plots
- 3 3. source contribution / lambda trace plots
- 4 4. fallback reason timeline
- 5 5. scenario-region labels
- 6 6. LiDAR degeneracy diagnostics
- 7 7. trend-based validator checks
- 8 8. no-source negative test 提前执行

你最终的实验结论应该按这四层写:

代码块

- 1 1. GNSS advisory predictor correctness
- 2 2. Predictor fallback safety
- 3 3. LiDAR diagnostic explainability
- 4 4. Fusion mathematical consistency

先不要写:

代码块

- 1 planner avoids high-risk areas
- 2 trajectory becomes safer
- 3 B-spline cost works

那些属于下一阶段 Safety Grid / Planner Adapter 测试。